

DocuMind Enterprise: Sistema Inteligente de Gestión y Análisis Documental

Fase 02: Diseño

Juan Esteban Rojas Melendez

Universitaria Tecnológica de Santander

Tabla de Contenidos

1. Descripción del Problema y Contexto Empresarial
2. Identificación de la Necesidad y Oportunidad de Negocio
3. Objetivos del Proyecto
4. Alcance y Exclusiones
5. Identificación de Actores y Perfil de Usuarios
6. Perfil de Usuarios / Personas
7. Requerimientos Funcionales (RF)
8. Requerimientos No Funcionales (RNF)
9. Reglas de Negocio (RN)
10. Historias de Usuario (con Criterios de Aceptación)
11. Casos de Uso (Especificación Resumida)
12. Priorización de Requisitos (Técnica MoSCoW)
13. Análisis y Matriz de Riesgos del Proyecto
14. Matriz de Trazabilidad Inicial

1. Decisiones Tecnológicas y su Justificación

Para satisfacer las necesidades de extracción, clasificación y búsqueda semántica, se ha definido el siguiente *stack* tecnológico:

- **Frontend:** React.js con Tailwind CSS. Justificación: Permite la creación de interfaces reactivas para el dashboard y el módulo de chat en tiempo real.
- **Backend / API:** FastAPI (Python). Justificación: Python es el estándar de la industria para IA. FastAPI ofrece un alto rendimiento, tipado estático y generación automática de documentación REST (Swagger).
- **Base de Datos y Almacenamiento:** PostgreSQL con la extensión pgvector. Justificación: Centraliza la persistencia relacional (usuarios, metadatos) y vectorial (fragmentos de texto) en un solo motor, evitando la complejidad de mantener dos bases de datos separadas.
- **Inteligencia Artificial (IA):** API de Google Gemini (**gemini-3.6-flash** para texto/JSON estructurado y **gemini-embedding-2** para vectores semánticos)). Justificación: Ofrece alto rendimiento sin costo inicial (capa gratuita) y capacidad nativa de estructurar salidas en formato JSON.
- **Contenedorización e Infraestructura:** Docker y Docker Compose. *Justificación:* Se utiliza para aislar el motor de base de datos PostgreSQL y la extensión pgvector en un contenedor estandarizado (pgvector/pgvector:pg16). Esto garantiza que el proyecto sea 100% reproducible en cualquier máquina sin tener que compilar extensiones a nivel del sistema operativo, evitando conflictos de dependencias y asegurando la persistencia de datos mediante volúmenes.
- **Gestión Dinámica de Versiones de IA:** La integración con Google AI Studio se implementa mediante variables de entorno en FastAPI (GEMINI_TEXT_MODEL y

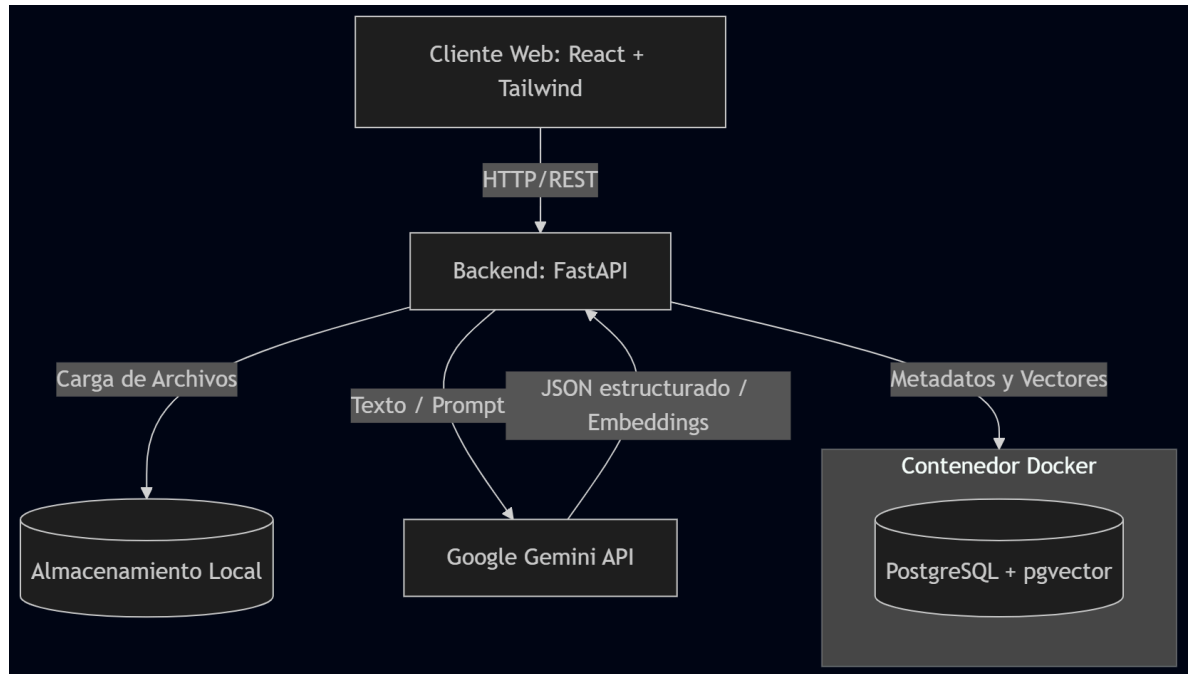
GEMINI_EMBEDDING_MODEL). Se adopta como base activa el modelo multimodal gemini-3.6-flash para tareas de clasificación, síntesis y extracción JSON, y el modelo gemini-embedding-2 para la indexación vectorial. Esta arquitectura desacoplada aísla la lógica de negocio ante las políticas de ciclo de vida y deprecación de modelos del proveedor, manteniendo la consistencia dimensional del índice HNSW en PostgreSQL mediante el ajuste forzado de `output_dimensionality=768`

2. Arquitectura General de la Solución

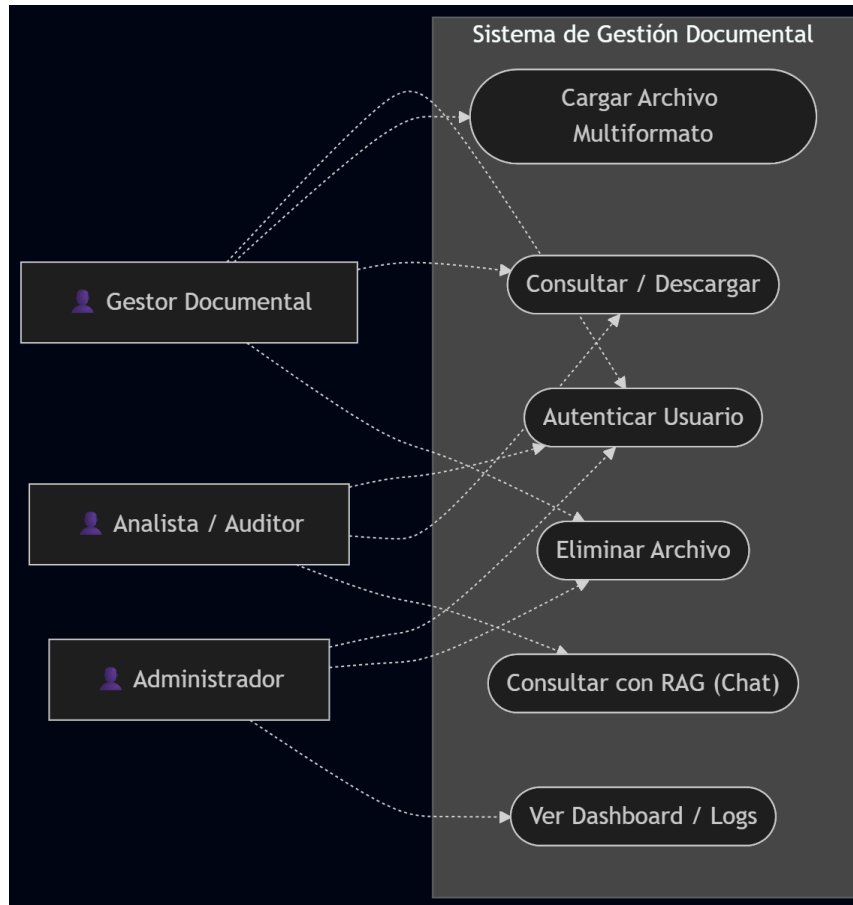
La plataforma se basa en una arquitectura Cliente-Servidor separada en capas:

- **Capa de Presentación (Frontend):** Aplicación de página única (SPA) que consume los servicios REST.
- **Capa de Lógica de Negocio (Backend):** Servidor FastAPI que orquesta la autenticación, la recepción de archivos (almacenados en un volumen local) y la comunicación asíncrona con los servicios de IA.
- **Capa de Datos:** PostgreSQL para persistencia de metadatos y vectores.
- **Capa de Inteligencia Artificial:** Integración externa con los servicios de Google Gemini para NLP y generación de *embeddings*.
- **Capa de Datos:** PostgreSQL para persistencia de metadatos y vectores, encapsulado dentro de un contenedor de Docker. El almacenamiento físico se garantiza mediante volúmenes de Docker (`documind_postgres_data`) para que la información persista incluso si el contenedor se reinicia o destruye.

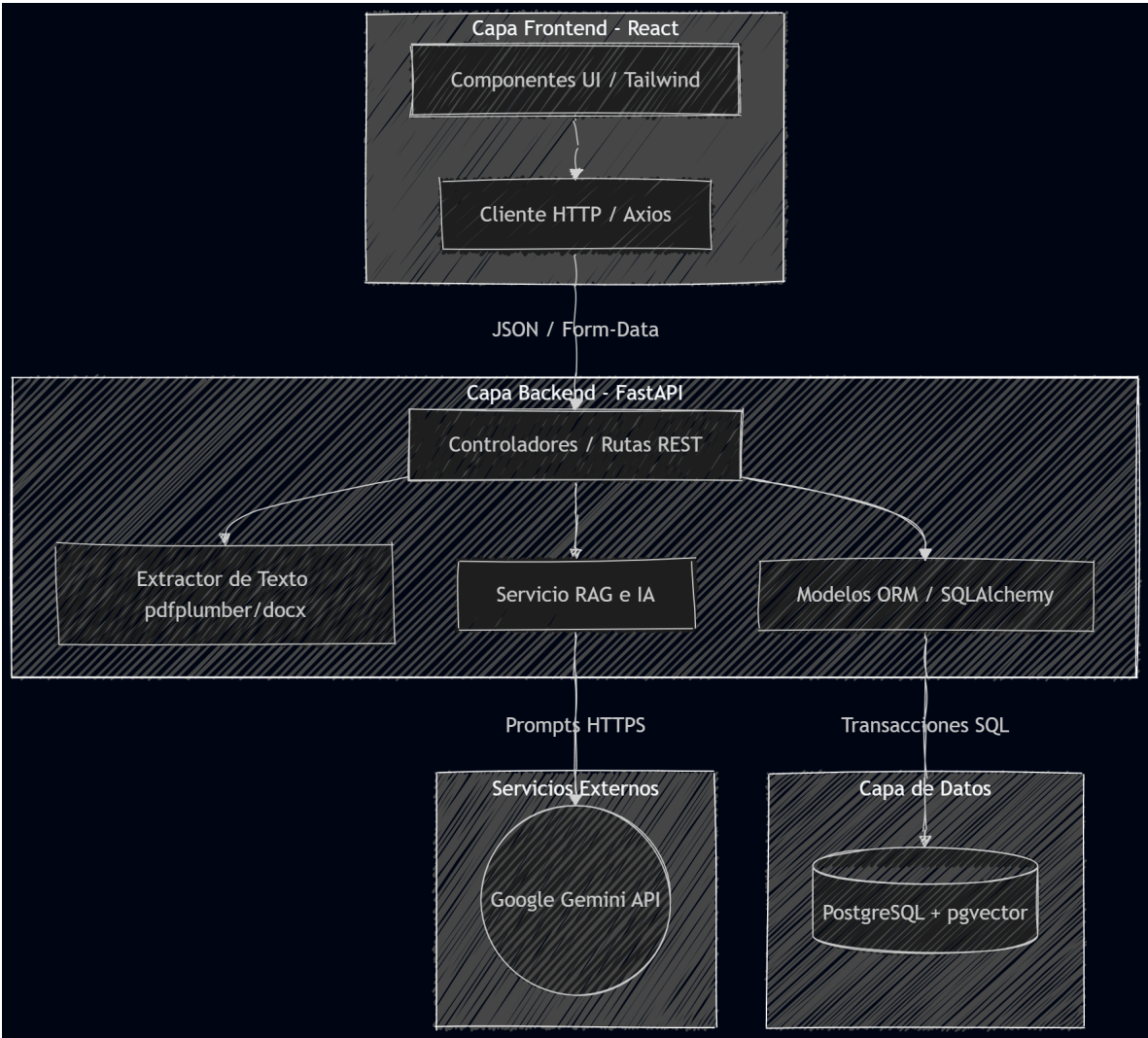
3. Diagrama de Arquitectura



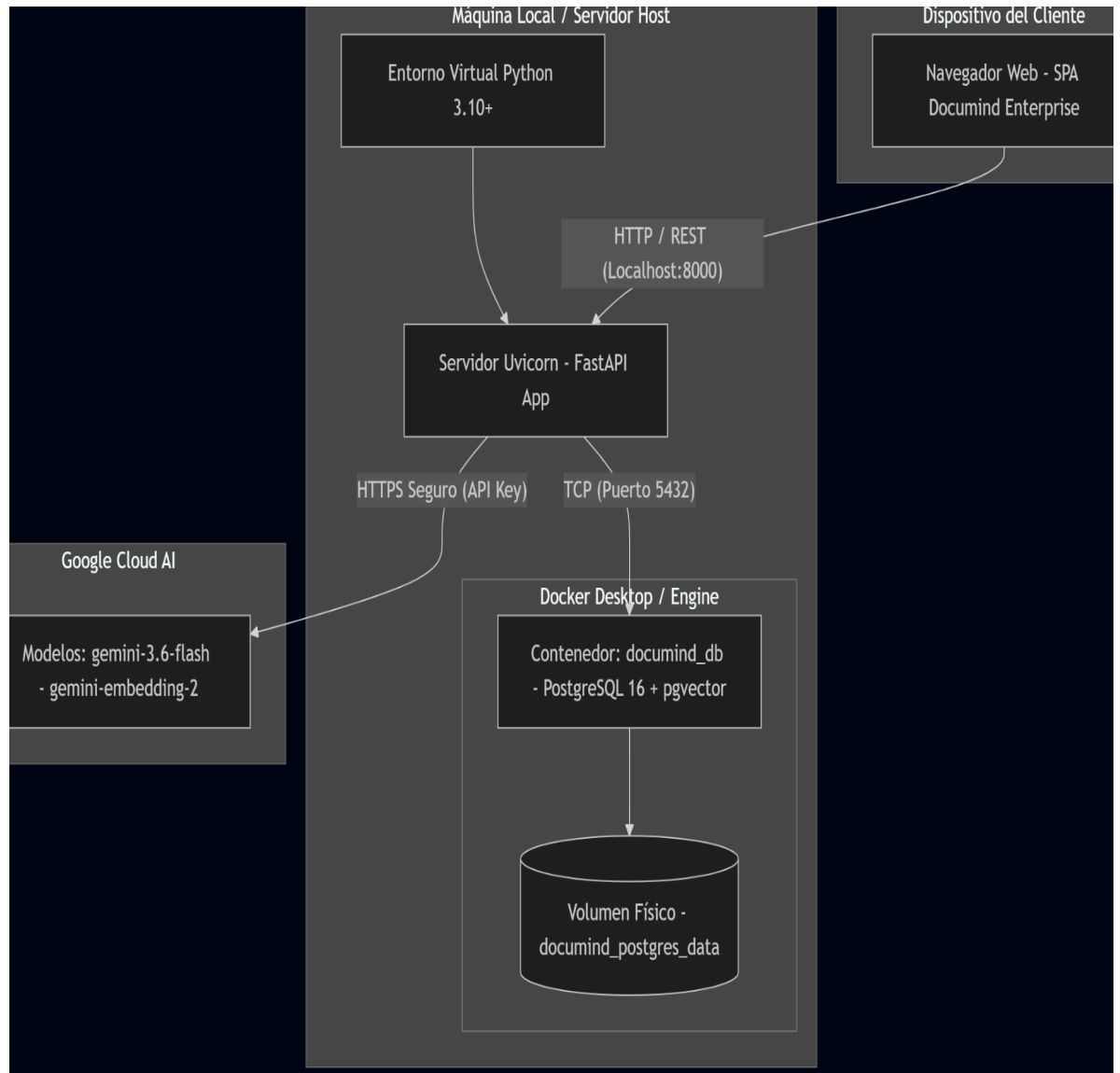
4. Diagrama de Casos de Uso



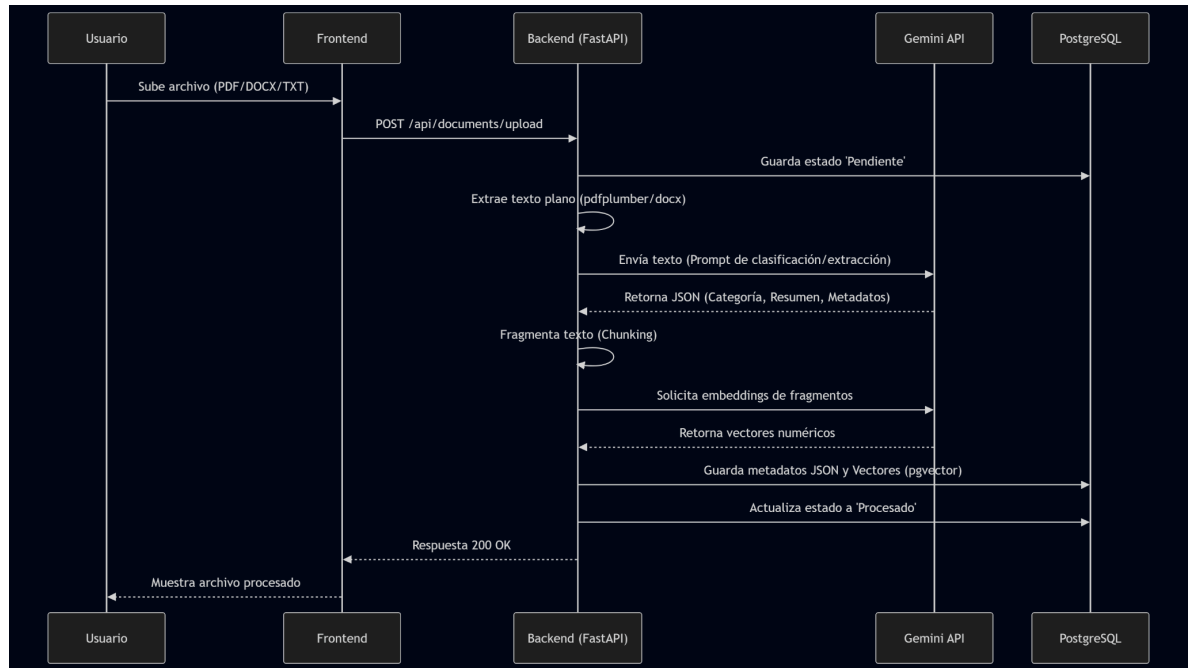
5. Diagrama de Componentes



6. Diagrama de Despliegue



7. Diagrama de Secuencia de Procesamiento Documental



8. Modelo Entidad-Relación y Diccionario de Datos

Se implementa un modelo de datos híbrido (Relacional + Vectorial).

Se configura explícitamente el parámetro `output_dimensionality=768` en las llamadas a `gemini-embedding-2`, garantizando compatibilidad nativa con la longitud vectorial de 768 dimensiones definida en el esquema de PostgreSQL / `pgvector`.

9. Diseño de API / Servicios

La comunicación entre frontend y backend se expone a través de endpoints REST protegidos:

Metodo	Endpoint	Proposito	Payload / Body
POST	/api/auth/login	Autenticacion de usuario	{ "email": "...", "password": "..." }
GET	/api/dashboard/stats	Indicadores del repositorio	Token JWT en Header

POST	/ap/documents/upload	Carga multiformato	Form-Data: file (.pdf, .docx, .txt)
GET	/api/documents/{id}	Consulta de metadatos y JSON	Parámetro de ruta id
DELETE	/api/documents/{id}	Eliminacion en cascada	Parámetro de ruta id
POST	/api/rag/query	Pregunta en lenguaje natural	{ "query": "¿Fecha del contrato?" }

10. Flujo de Procesamiento Documental e Integración con IA

**Archivo → Extracción de contenido → Procesamiento mediante IA → Análisis
→ Almacenamiento de resultados → Búsqueda y consulta → Respuesta**

1. **Extracción:** Se usa pdfplumber y python-docx para obtener texto plano.
2. **Análisis Estructurado:** Se envía a Gemini con un *System Prompt* que obliga una salida JSON (Structured Outputs) con la categoría, el resumen y las entidades.
3. **Integración RAG:** El texto se corta en fragmentos de ~500 caracteres, se vectorizan y se indexan en pgvector. Cuando el usuario consulta, se extraen los fragmentos mediante similitud de coseno y se insertan en el contexto del LLM para formular la respuesta.

11. Diseño de Interfaces y Prototipos

La interfaz de usuario contará con las siguientes vistas principales construidas en React:

- **Vista Login:** Formulario centralizado con validación de credenciales.
- **Dashboard (Inicio):** Tarjetas de métricas (Total archivos, % procesados) y gráficas de barras por categoría documental.

- **Gestor de Repositorio (CRUD):** Tabla con la lista de documentos, estado visual (colores semánticos), botón de carga (drag & drop) y acciones (ver, eliminar).
- **Vista de Detalle:** Pantalla dividida; a la izquierda los metadatos (Categoría, Resumen, JSON extraído), a la derecha opciones de descarga.
- **Consola de Búsqueda (RAG):** Interfaz tipo chat donde el usuario escribe su pregunta y el sistema responde citando el nombre del archivo fuente.

12. Diseño Básico de Seguridad

- **Autenticación:** Implementación de JSON Web Tokens (JWT) con expiración corta; el token viaja en el encabezado Authorization: Bearer.
- **Protección de Secretos:** Las claves de base de datos y la GEMINI_API_KEY residirán exclusivamente en el archivo .env del servidor, ignorado en Git.
- **Sanitización:** Los archivos cargados serán validados por su tipo MIME real (Magic Numbers), no solo por su extensión, previniendo subidas maliciosas. CORS (Cross-Origin Resource Sharing) estará estrictamente configurado para aceptar peticiones solo desde el dominio del frontend.